TEAM 5 · ONLINE SHOPPING SYSTEM

FINAL SHARED ARCHITECTURE AND API CONTRACT

Version 2.0 · Member 1 / Member 2 Alignment Draft

Document item	Details
Prepared by	Mohammed Owais — Member 1 (Core system, customer journey, payment and integration)
Collaborator	Rashika — Member 2 (Seller operations, inventory, coupons, delivery and returns operations)
Backend / database	Java 21 · Spring Boot 3.4.x · Spring Data JPA · MySQL 8
Frontend	HTML · CSS · JavaScript
Repository branch	feature/owais-core (shared foundation); feature/rashika-operations (operations module)
Document status	Consolidated Version 2.0 — prepared from Rashika's review; requires final confirmation by both members
Implementation status	Design only. No repository implementation, commit, push or merge is authorized by this document.

Purpose. Establish one shared data model, lifecycle rules, security boundary and API contract before either member implements dependent modules.

This document incorporates Rashika's decisions and adds explicit technical rules where needed to make the contracts implementable. Any item marked “confirm” remains an open joint decision and must not be silently treated as approved.

Document guide

1. Scope, ownership and architecture principles
2. Shared entity and database contract
3. Status enums and permitted transitions
4. Inventory reservation, commit and concurrency rules
5. Coupon calculation and item-level discount allocation
6. Seller fulfilment and shipment/consignment model
7. Cancellation, returns and refunds
8. Role permissions and ownership enforcement
9. REST API contracts and JSON examples
10. Notifications and auditability
11. Error handling, idempotency and transaction rules
12. Member responsibilities, integration sequence and sign-off

Key decisions incorporated from Rashika

- Use MySQL JSON for product specifications; calculate ratings from customer reviews.
- Use a 15-minute stock reservation; commit only after verified payment confirmation; expire/release automatically.
- Use OrderItem-level PROCESSING; keep RESERVED exclusively in StockReservation.
- Create one Shipment per seller package, with one delivery agent per shipment and OTP-based delivery completion.
- Allocate coupon discount proportionally to order items and store each item's allocated discount and net paid amount.
- Restock only after seller inspection confirms the returned/cancelled item is eligible for resale.
- Use configurable seven-day return eligibility; prevent duplicate refund processing.
- Derive parent order status from item-level progress and notify customers of material lifecycle changes.

1. Scope, ownership and architecture principles

The application is one integrated multi-seller online shopping system. A customer can browse products, maintain a cart, reserve stock at checkout, apply a server-validated coupon, pay, and track an order. Sellers manage only their own catalogue and order items. Admins assign shipments to delivery agents. Delivery agents update only shipments assigned to them. Returns and refunds are tracked independently.

Area	Primary owner	Shared dependency / boundary
Core foundation, authentication, customer journey, payment, order creation, customer order tracking	Owais (Member 1)	Shared User, Product, Order, OrderItem, StockReservation, Shipment contract, security and common response format
Seller dashboard, seller inventory operations, coupon management, fulfilment, shipment operations, delivery dashboard, seller return inspection	Rashika (Member 2)	Must use shared entities, role checks, lifecycle transitions and agreed APIs; no duplicate core models
Shared integration decisions	Both	Entity fields, enums, transitions, API request/response formats and permissions require joint agreement

Architecture rules

- Backend is authoritative for price, stock, coupon eligibility, discounts, payable totals, ownership and status transitions. Never trust client-supplied totals, role IDs or seller IDs.

- Use DTOs at API boundaries; do not expose JPA entities directly.
- Use BigDecimal / DECIMAL(12,2) for money. Never use floating-point arithmetic for currency.
- Use UTC timestamps in storage and ISO-8601 timestamps with timezone in API responses.
- Use database transactions for multi-row stock, order, coupon-usage and refund operations.
- Use foreign keys, unique constraints and indexes for ownership references, codes and idempotency keys.
- Do not commit secrets, payment keys, database passwords, OTP plaintext or internal service credentials.

2. Shared entity and database contract

The following is the proposed MVP schema contract. Field names are canonical API/domain names; implementation may use Java camelCase and snake_case database columns consistently.

2.1 User — users

Field	Type / rule	Purpose
id	BIGINT PK AUTO_INCREMENT	Internal identity
email	VARCHAR(150), NOT NULL, UNIQUE	Login identity; normalize case
password_hash	VARCHAR(255), NOT NULL	BCrypt hash only; never return
full_name	VARCHAR(120), NOT NULL	Display/contact name
phone_number	VARCHAR(20), nullable or required per registration validation	Contact number
role	ENUM/string: CUSTOMER, SELLER, DELIVERY_AGENT, ADMIN	Single MVP role per account
is_active	BOOLEAN NOT NULL DEFAULT TRUE	Account activation
created_at / updated_at	DATETIME(6)	Audit timestamps

2.2 Product — products

Field	Type / rule	Purpose
id	BIGINT PK	Product identity
seller_id	BIGINT FK users.id, NOT NULL, indexed	Owning seller; immutable except admin-controlled transfer
name / brand / category	VARCHAR(200/100/100), NOT NULL	Search and classification
description	TEXT, nullable	Product details
specifications	JSON, nullable	Flexible technical attributes
price	DECIMAL(12,2), NOT NULL, >= 0	Current selling price
available_stock	INT NOT NULL, >= 0	Sellable units not held or committed
reserved_stock	INT NOT NULL DEFAULT 0, >= 0	Units currently held by active reservations
is_active	BOOLEAN DEFAULT TRUE	Catalogue visibility
created_at / updated_at	DATETIME(6)	Audit timestamps

Ratings: do not permit sellers to set average_rating or review_count manually. Add ProductReview (review_id, product_id, customer_id, order_item_id, rating 1–5, review_text, status, created_at). Enforce one eligible review per delivered OrderItem. Compute the average and count from approved reviews; a cached aggregate may be maintained transactionally.

2.3 StockReservation and StockReservationItem

Entity / field	Type / rule	Purpose
StockReservation.id	BIGINT PK	Reservation header
reservation_code	VARCHAR(64), UNIQUE	Public-safe reservation reference

Entity / field	Type / rule	Purpose
checkout_reference	VARCHAR(64), NOT NULL, UNIQUE per customer or globally unique	Idempotent checkout reference
customer_id	BIGINT FK users.id	Derived from authenticated customer
status	ACTIVE, COMMITTED, RELEASED, EXPIRED	Reservation lifecycle
expires_at / created_at	DATETIME(6)	15-minute TTL and audit
StockReservationItem.id	BIGINT PK	Reservation line
reservation_id	BIGINT FK, NOT NULL	Header link
product_id	BIGINT FK products.id	Reserved product
quantity	INT > 0	Held quantity
unit_price_snapshot	DECIMAL(12,2)	Price captured at checkout, if reservation locks price

A reservation is a header with one or more item rows. The customer identity comes from the security context. The server calculates and validates each product's current price and stock. The 15-minute expiry is set server-side.

2.4 Order — orders

Field	Type / rule	Purpose
id / order_number	BIGINT PK; VARCHAR(64) UNIQUE	Internal and customer-facing identifiers
customer_id	BIGINT FK users.id, NOT NULL	Order owner
subtotal_amount	DECIMAL(12,2)	Sum of item gross totals before coupon discount
discount_amount	DECIMAL(12,2) DEFAULT 0	Validated coupon discount
shipping_fee	DECIMAL(12,2) DEFAULT 0	Order shipping charge
total_amount	DECIMAL(12,2)	Final amount charged = subtotal - discount + shipping
coupon_id / coupon_code_snapshot	FK nullable; VARCHAR(50) nullable	Coupon reference and immutable code snapshot
order_status / payment_status	Enums defined in Section 3	Derived order lifecycle and financial state
payment_method	UPI, CARD, MOCK, COD (MVP scope to confirm)	Payment method
shipping_address_snapshot	JSON NOT NULL	Immutable recipient/address snapshot
created_at / updated_at	DATETIME(6)	Audit timestamps

2.5 OrderItem — order_items

Field	Type / rule	Purpose
id	BIGINT PK	Line identity
order_id	BIGINT FK orders.id	Parent order
seller_id	BIGINT FK users.id	Seller ownership snapshot
product_id	BIGINT FK products.id	Catalogue reference
product_name_snapshot	VARCHAR(200)	Name at purchase time
quantity	INT > 0	Purchased quantity
unit_price_snapshot	DECIMAL(12,2)	Immutable unit price at purchase
gross_amount	DECIMAL(12,2)	quantity × unit_price_snapshot
allocated_discount_amount	DECIMAL(12,2) DEFAULT 0	Coupon discount allocated to this line
net_paid_amount	DECIMAL(12,2)	gross_amount – allocated_discount_amount; excludes separately handled shipping

Field	Type / rule	Purpose
item_status	PENDING_SELLER_ACCEPTANCE, PROCESSING, REJECTED, PACKED, DISPATCHED, DELIVERED, CANCELLED	Fulfilment lifecycle
delivered_at	DATETIME(6), nullable	Return-window calculation
created_at / updated_at	DATETIME(6)	Audit timestamps

Do not store delivery_agent_id on OrderItem as the authoritative assignment once Shipment exists. The authoritative courier assignment belongs to Shipment. An optional read-only projection may be returned in DTOs.

2.6 Shipment and ShipmentItem

Field	Type / rule	Purpose
Shipment.id / shipment_code	BIGINT PK; VARCHAR(64) UNIQUE	Seller package identity
order_id	BIGINT FK orders.id	Parent customer order
seller_id	BIGINT FK users.id	Exactly one seller per shipment
delivery_agent_id	BIGINT FK users.id, nullable	One assigned agent per shipment
status	UNASSIGNED, ASSIGNED, PICKED_UP, IN_TRANSIT, OUT_FOR_DELIVERY, DELIVERY_FAILED, RETURN_TO_SELLER, RETURNED_TO_SELLER, DELIVERED	Package-level delivery lifecycle
tracking_number	VARCHAR(100), nullable	Tracking reference
delivery_otp_hash / otp_expires_at / otp_attempts	Secure hash, timestamp, integer	OTP verification; never store or return plaintext
picked_up_at / delivered_at	DATETIME(6), nullable	Shipment milestones
failure_reason / notes	VARCHAR/TEXT, nullable	Operational notes
created_at / updated_at	DATETIME(6)	Audit timestamps
ShipmentItem.id	BIGINT PK	Join row
shipment_id	BIGINT FK shipments.id	Package reference
order_item_id	BIGINT FK order_items.id, UNIQUE	Each order item assigned to at most one active shipment

MVP grouping rule: create one Shipment for each seller's accepted/packed items within an order. A shipment must contain only OrderItems with the same order_id and seller_id. One delivery agent is assigned to each Shipment. If later partial dispatches by the same seller are needed, allow multiple shipments per seller/order and remove the one-shipment-per-item uniqueness only through a reviewed schema change.

2.7 Coupon and CouponUsage

Field	Type / rule	Purpose
Coupon.id / code	BIGINT PK; VARCHAR(50) UNIQUE	Coupon identity/code
discount_type	PERCENTAGE or FIXED_AMOUNT	Discount calculation mode
discount_value	DECIMAL(12,2), > 0	Percent or fixed amount
minimum_order_amount	DECIMAL(12,2) DEFAULT 0	Eligibility threshold
maximum_discount_amount	DECIMAL(12,2), nullable	Cap for percentage discounts
starts_at / expires_at	DATETIME(6)	Validity window
total_usage_limit	INT nullable	Global redemption cap
per_customer_usage_limit	INT DEFAULT 1	Per-user redemption cap
usage_count	INT DEFAULT 0	Concurrency-safe global count/cache
is_active	BOOLEAN DEFAULT TRUE	Admin activation

Field	Type / rule	Purpose
CouponUsage.id	BIGINT PK	Redemption record
coupon_id / customer_id / order_id	FKs; order_id unique for one applied coupon per order	Usage ownership and audit
discount_amount / used_at	DECIMAL(12,2); DATETIME(6)	Actual redeemed discount and timestamp

2.8 ReturnRequest and RefundTransaction

Entity / field	Type / rule	Purpose
ReturnRequest.id / return_code	BIGINT PK; VARCHAR(64) UNIQUE	Return reference
order_item_id / customer_id / seller_id	FKs, NOT NULL	Line, customer and seller ownership
quantity	INT > 0	Requested return quantity
reason / description	VARCHAR(100); TEXT nullable	Customer reason
status	REQUESTED, APPROVED, REJECTED, PICKUP_SCHEDULED, RECEIVED, INSPECTED, COMPLETED	Return workflow
eligibility_policy_snapshot	JSON or fields	Eligibility rule applied at purchase/delivery
inspection_condition / restockable	Enum/string; BOOLEAN nullable	Seller/admin inspection result
RefundTransaction.id / refund_code	BIGINT PK; VARCHAR(64) UNIQUE	Refund record
order_id / order_item_id / return_request_id	FKs; return_request_id nullable for cancellation	Financial linkage
amount / status	DECIMAL(12,2); INITIATED, PROCESSING, COMPLETED, FAILED	Refund amount and processing state
idempotency_key	VARCHAR(100) UNIQUE	Prevents duplicate refund execution
gateway_reference / created_at	VARCHAR(100); DATETIME(6)	Gateway audit

2.9 Notification and audit records

Entity	Minimum fields	Purpose
Notification	id, user_id, event_type, title, message, reference_type, reference_id, read_at, created_at	In-app customer/seller/agent notifications
StatusHistory / AuditEvent	id, aggregate_type, aggregate_id, from_status, to_status, actor_id, reason, metadata_json, created_at	Trace status changes and actor; useful for dispute/debugging

3. Status enums and permitted transitions

3.1 Order status — derived, not directly edited by sellers

Status	Meaning / derivation
CREATED	Order record created before payment is complete
PAYMENT_PENDING	Payment initiation/confirmation is pending
PROCESSING	Payment captured and all active items accepted/being processed
PARTIALLY_CONFIRMED	At least one item is PROCESSING and at least one item is still awaiting decision or has been rejected/cancelled
PARTIALLY_SHIPPED	At least one item/shipment dispatched and at least one eligible item not yet dispatched
SHIPPED	All non-cancelled/non-rejected items have been dispatched
PARTIALLY_DELIVERED	At least one item delivered and at least one non-cancelled item remains undelivered
DELIVERED	All non-cancelled/non-rejected items delivered
PARTIALLY_CANCELLED	At least one item cancelled/rejected and at least one item remains active or delivered

Status	Meaning / derivation
CANCELLED	No fulfillable items remain; all items rejected/cancelled
COMPLETED	Post-delivery lifecycle closed; no pending eligible return/refund actions

Aggregation precedence must be implemented centrally in OrderStatusService and covered by tests. If more than one partial condition is true, use this priority: PARTIALLY_DELIVERED, PARTIALLY_SHIPPED, PARTIALLY_CANCELLED, PARTIALLY_CONFIRMED. Payment failure/cancellation is handled separately by payment_status and does not fabricate a fulfilment status.

3.2 OrderItem status

Transition	Actor / condition
PENDING_SELLER_ACCEPTANCE → PROCESSING	Owning seller accepts after payment is verified
PENDING_SELLER_ACCEPTANCE → REJECTED	Owning seller rejects with reason; refund workflow begins
PROCESSING → PACKED	Owning seller marks item packed
PACKED → DISPATCHED	Seller performs handover/dispatch action; shipment must be assigned and pickup confirmed
DISPATCHED → DELIVERED	System updates after assigned delivery agent verifies OTP on Shipment
PENDING_SELLER_ACCEPTANCE or PROCESSING → CANCELLED	Customer cancellation accepted before packing/dispatch, subject to policy
PACKED / DISPATCHED / DELIVERED	No direct customer cancellation; use shipment/return workflow

RESERVED is not an OrderItem status. It exists only as StockReservation status ACTIVE. PROCESSING means seller acceptance has been recorded and fulfilment work has started.

3.3 Payment, reservation, shipment, return and refund states

Lifecycle	Allowed progression / rules
Payment	PENDING → AUTHORIZED → CAPTURED; PENDING/AUTHORIZED → FAILED. CAPTURED may later become PARTIALLY_REFUNDED or REFUNDED. A verified gateway webhook is authoritative.
StockReservation	ACTIVE → COMMITTED after verified payment; ACTIVE → RELEASED on customer/system abort; ACTIVE → EXPIRED by scheduler. Terminal states cannot be reversed.
Shipment	UNASSIGNED → ASSIGNED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. From PICKED_UP/IN_TRANSIT/OUT_FOR_DELIVERY, a failed attempt may move to DELIVERY_FAILED; retry returns to IN_TRANSIT/OUT_FOR_DELIVERY. Return flow: DELIVERY_FAILED → RETURN_TO_SELLER → RETURNED_TO_SELLER.
ReturnRequest	REQUESTED → APPROVED or REJECTED; APPROVED → PICKUP_SCHEDULED → RECEIVED → INSPECTED → COMPLETED. No refund before inspection/approval, except approved pre-dispatch cancellation path.
RefundTransaction	INITIATED → PROCESSING → COMPLETED or FAILED. Retrying a failed gateway operation reuses the same idempotency key; never create a second refund for the same refundable amount.

OTP: Generate a time-limited OTP for the customer, store only a secure hash, rate-limit attempts, and verify it server-side before Shipment becomes DELIVERED. A delivery agent cannot submit DELIVERED without successful OTP verification.

Clarification: Shipment status is authoritative for package tracking. OrderItem status is updated from shipment events for customer order aggregation; clients cannot independently mutate either status.

4. Inventory reservation, commit and concurrency

4.1 Stock arithmetic

The model uses available_stock for unreserved sellable quantity and reserved_stock for active holds. On reservation, available_stock decreases and reserved_stock increases. On successful payment commit, reserved_stock decreases while available_stock remains unchanged. On release/expiry, reserved_stock decreases and available_stock increases.

Event	available_stock	reserved_stock	Reservation
Reserve quantity q	−q	+q	ACTIVE
Commit after verified payment	No change	−q	COMMITTED
Release / expire active hold	+q	−q	RELEASED / EXPIRED
Seller rejection after commit	+q	No change	Already COMMITTED; stock returned through inventory service
Eligible pre-dispatch cancellation	+q	No change	Already COMMITTED; stock returned through inventory service
Return inspected and restockable	+q	No change	Return completed; stock restored once

4.2 Required implementation controls

• Reserve all requested products atomically: lock product rows (SELECT ... FOR UPDATE) in deterministic product-ID order, validate all quantities, then update stock and create reservation rows in one transaction.

• Reject if any requested quantity exceeds available_stock; roll back the entire reservation. Return HTTP 409 with the affected product and available quantity.

• Use checkout_reference and reservation_code uniqueness for idempotency. Repeating the same reserve request returns the existing active reservation rather than decrementing stock again.

• Scheduled expiry job finds ACTIVE reservations with expires_at <= now, locks each reservation and product rows, releases exactly once, and marks EXPIRED.

• POST /api/inventory/commit is internal-only and idempotent. It accepts a reservation reference and verified payment reference; it must not be callable by a browser/customer directly.

• Commit only after payment signature/webhook verification. Duplicate payment webhooks return the already committed result.

• All stock restoration paths call one InventoryService method with a unique stock-movement reference. Sellers cannot restore stock by editing OrderItem status.

• Seller stock adjustments require ownership checks, non-negative quantities, and an audit record. Never allow available_stock to fall below zero.

5. Coupon calculation and item-level discount allocation

5.1 Coupon validation

• Coupon must exist, be active, and current time must be within starts_at and expires_at.

• Cart subtotal is calculated by the backend from the authenticated customer's active cart and current product prices.

• Subtotal must meet minimum_order_amount. Enforce maximum_discount_amount, total_usage_limit and per_customer_usage_limit.

• Support PERCENTAGE and FIXED_AMOUNT. Discount cannot exceed eligible subtotal.

• Reserve or increment coupon usage atomically when order placement is confirmed; release the usage reservation if payment fails or order is abandoned, according to the agreed checkout policy.

• Coupon management is ADMIN-owned for MVP unless both members explicitly approve seller-scoped coupons.

5.2 Discount allocation

At order creation, allocate the validated order-level coupon discount proportionally across eligible OrderItems by each item's gross_amount. Persist allocated_discount_amount and net_paid_amount on every OrderItem. Use decimal rounding to two places; allocate any rounding remainder deterministically to the final eligible line so the sum of item allocations exactly equals the order discount.

```
line_weight = item.gross_amount / order.subtotal_amount
raw_line_discount = order.discount_amount * line_weight
allocated_discount_amount = round_to_2dp(raw_line_discount)
net_paid_amount = gross_amount - allocated_discount_amount

Invariant:
```

```
SUM(order_items.allocated_discount_amount) = order.discount_amount
SUM(order_items.net_paid_amount) = order.subtotal_amount - order.discount_amount
```

For a partial return of quantity q from a line with original quantity Q, base refundable amount = stored net_paid_amount × q / Q, rounded to currency precision. Do not recalculate the coupon because the remaining basket later falls below the original threshold. Shipping-fee refunds, taxes, payment-gateway fees and inspection deductions are not fully specified by the current agreement; keep them as explicit policy settings and obtain joint approval before implementing them.

6. Seller fulfilment and shipment/consignment model

6.1 Seller workflow

- Seller views only OrderItems whose seller_id equals the authenticated seller ID.
- Seller accepts an eligible item: PENDING_SELLER_ACCEPTANCE → PROCESSING.
- Seller rejects an item with a reason. The system invokes the shared refund and inventory services; seller does not directly mutate stock or create gateway refunds.
- Seller packs items. PACKED is allowed only from PROCESSING.
- System creates/updates one Shipment per seller package and links the seller's packed OrderItems through ShipmentItem.
- Admin assigns one delivery agent to each Shipment.
- Seller performs an explicit dispatch/handover action. Shipment moves to PICKED_UP only when handover is confirmed; linked OrderItems become DISPATCHED.
- Delivery agent updates package tracking through permitted transitions; OTP verification is required to complete delivery.

6.2 Shipment example

```
{
  "shipmentCode": "SHP-20261108-1007",
  "orderId": 501,
  "sellerId": 22,
  "deliveryAgentId": 8,
  "status": "ASSIGNED",
  "trackingNumber": "TRK-884421",
  "items": [
    { "orderItemId": 402, "quantity": 1 },
    { "orderItemId": 403, "quantity": 2 }
  ]
}
```

The same shipment must never contain items from different sellers or different parent orders. Delivery agents see only shipments assigned to their authenticated user ID.

7. Cancellation, returns and refunds

7.1 Cancellation

- Customer can request cancellation for the whole order or selected OrderItems, but the server evaluates each line independently.
- Cancellation is permitted before dispatch; recommended MVP cutoff is before PACKED. If the seller has already packed an item, cancellation is rejected or requires seller/admin exception approval.
- After dispatch, customer must use the return workflow rather than cancellation.
- Accepted cancellation marks the relevant OrderItem CANCELLED, invokes inventory restoration once, and creates/updates an idempotent RefundTransaction for the actual net paid amount.
- Seller rejection after payment follows the same item-level refund and stock-restoration services; unaffected seller items continue fulfilment.

7.2 Returns

- Customer creates a return request for a delivered OrderItem and a quantity not exceeding delivered quantity minus already returned quantity.

- Default return window is seven calendar days from delivered_at. Eligibility can be configured by product/category; excluded categories and non-returnable products must be explicit in catalogue/policy.
- Seller/admin reviews the request. Approved returns are scheduled for pickup, received, and inspected.
- Inventory is restocked only after inspection confirms RESTOCKABLE. A non-restockable return is not added to available_stock.
- Refund amount is based on stored item net paid amount for the returned quantity, not current product price or original list price.
- ReturnRequest and RefundTransaction remain separate records. Each refund references its return request and uses a unique idempotency key.

7.3 Refund calculation and safeguards

```
refundable_per_unit = order_item.net_paid_amount / order_item.quantity
base_refund = refundable_per_unit * approved_return_quantity
final_refund = apply_approved_policy_adjustments(base_refund)

Safeguards:
- cumulative refunds for an OrderItem must not exceed its net_paid_amount;
- each ReturnRequest can be refunded only once for its approved quantity;
- gateway retries reuse the same idempotency key;
- seller inspection does not directly mark a gateway refund as COMPLETED;
- only verified gateway response/webhook completes the financial refund.
```

Policy items still requiring explicit joint approval: shipping-fee refund rules, tax treatment, whether any inspection deduction is allowed and who can authorize it, and whether COD refunds are recorded as manual bank/UPI payouts or simulated in the MVP.

8. Role permissions and ownership enforcement

Role	Allowed actions	Forbidden / restricted
CUSTOMER	Browse; manage own cart; checkout; apply coupon; pay; view own orders; cancel eligible items; request returns; view own notifications	Cannot supply customer_id, change prices/stock/status, access another customer's order or trigger internal inventory commit
SELLER	Manage own products; view own inventory; update own stock; view/accept/reject/pack own OrderItems; initiate package handover; review/inspect own returns	Cannot read another seller's items, assign delivery agents, directly refund, or directly mutate stock via status update
DELIVERY_AGENT	View own assigned Shipments; update allowed Shipment tracking states; verify delivery OTP	Cannot view unassigned shipments, change seller/order item decisions, or mark delivered without OTP
ADMIN	Manage users/roles, coupon lifecycle, assign agents, oversee shipments, handle authorized exceptions and policy settings	Sensitive actions must be audited; admin does not bypass idempotency or financial safeguards
INTERNAL SERVICE	Commit reservation after verified payment; process trusted gateway events and scheduled expiry	Not a human role; endpoint must be protected from public/browser access

Every protected query must apply ownership filtering in the database/service layer, not merely hide UI controls. Return 404 for inaccessible resource IDs where appropriate to avoid leaking another user's data.

9. REST API contracts and JSON examples

All APIs use the common response envelope in Section 11. The examples below show the canonical payload fields; timestamps and generated IDs are illustrative.

9.1 Inventory APIs — shared service boundary

Method / endpoint	Role	Purpose
GET /api/inventory/check?productId=101&quantity;=2	CUSTOMER, SELLER (own product), ADMIN	Check current purchasable stock; advisory only, not a reservation
GET /api/seller/inventory	SELLER	List only seller-owned products and stock
PATCH /api/seller/inventory/products/{productId}	SELLER	Set or adjust own product stock with reason and audit

Method / endpoint	Role	Purpose
POST /api/inventory/reserve	CUSTOMER	Create atomic 15-minute reservation
POST /api/inventory/release	CUSTOMER / ADMIN / internal expiry job	Release an active reservation exactly once
POST /api/inventory/commit	INTERNAL SERVICE ONLY	Commit active reservation after verified payment

```
POST /api/inventory/reserve
{
  "checkoutReference": "CHK-8849",
  "items": [
    { "productId": 101, "quantity": 2 },
    { "productId": 105, "quantity": 1 }
  ]
}

201 Created
{
  "success": true,
  "message": "Stock reserved",
  "data": {
    "reservationCode": "RES-882104",
    "status": "ACTIVE",
    "expiresAt": "2026-11-08T12:15:00Z",
    "items": [
      { "productId": 101, "quantity": 2 },
      { "productId": 105, "quantity": 1 }
    ]
  },
  "timestamp": "2026-11-08T12:00:00Z"
}

POST /api/inventory/commit  [internal authenticated call]
{
  "reservationCode": "RES-882104",
  "paymentReference": "PAY-GW-77881",
  "eventId": "gateway-event-9921"
}
```

```
PATCH /api/seller/inventory/products/101
{
  "newAvailableStock": 25,
  "reason": "PHYSICAL_STOCK_RECOUNT"
}

POST /api/inventory/release
{
  "reservationCode": "RES-882104",
  "reason": "CHECKOUT_ABORTED"
}
```

Seller stock API must not accept reserved_stock directly. The service computes stock deltas and rejects updates that would violate available/reserved invariants.

9.2 Coupon management and application APIs

Method / endpoint	Role	Purpose
POST /api/admin/coupons	ADMIN	Create coupon
PUT /api/admin/coupons/{couponId}	ADMIN	Update coupon rules
PATCH /api/admin/coupons/{couponId}/status	ADMIN	Activate/deactivate coupon
GET /api/admin/coupons	ADMIN	List/search coupons
POST /api/coupons/apply	CUSTOMER	Validate coupon against server-calculated cart

```
POST /api/admin/coupons
{
  "code": "FESTIVE25",
  "discountType": "PERCENTAGE",
  "discountValue": 25,
  "minimumOrderAmount": 500,
  "maximumDiscountAmount": 300,
```

```
  "startsAt": "2026-11-01T00:00:00Z",
  "expiresAt": "2026-11-30T23:59:59Z",
  "totalUsageLimit": 1000,
  "perCustomerUsageLimit": 1,
  "isActive": true
}

POST /api/coupons/apply
{ "couponCode": "FESTIVE25" }

200 OK
{
  "success": true,
  "message": "Coupon validated",
  "data": {
    "couponCode": "FESTIVE25",
    "discountType": "PERCENTAGE",
    "discountAmount": 125.00,
    "verifiedSubtotal": 500.00,
    "finalPayable": 375.00
  },
  "timestamp": "2026-11-08T12:00:00Z"
}
```

The apply endpoint accepts only couponCode. It does not accept customerId, sellerId, cartSubtotal, discountAmount or finalPayable from the client. Final validation and usage reservation are repeated atomically at order placement.

9.3 Seller fulfilment APIs

Method / endpoint	Role	Purpose
GET /api/seller/order-items?status=PENDING_SELLER_ACCEPTANCE	SELLER	List only own order items
PATCH /api/seller/order-items/{id}/decision	SELLER	Accept or reject with reason
PATCH /api/seller/order-items/{id}/pack	SELLER	Mark own item packed
POST /api/seller/shipments	SELLER	Create package from own packed items
POST /api/seller/shipments/{shipmentId}/handover	SELLER	Explicit dispatch/handover to assigned agent

```
PATCH /api/seller/order-items/402/decision
{ "decision": "ACCEPT" }

PATCH /api/seller/order-items/402/pack
{ "packageWeightKg": 2.4, "packagingNotes": "Fragile packaging" }

POST /api/seller/shipments
{
  "orderId": 501,
  "orderItemIds": [402, 403]
}

POST /api/seller/shipments/701/handover
{
  "handoverReference": "HANDOVER-2201",
  "notes": "Package handed to assigned agent"
}
```

The shipment-creation service validates that every selected item is PACKED, belongs to the authenticated seller and same order, and is not already assigned to another active shipment.

9.4 Admin and delivery-agent shipment APIs

Method / endpoint	Role	Purpose
GET /api/admin/shipments?status=UNASSIGNED	ADMIN	View packages awaiting assignment
POST /api/admin/shipments/{shipmentId}/assign	ADMIN	Assign one delivery agent
GET /api/delivery/shipments?status=ASSIGNED	DELIVERY_AGENT	List only own assigned shipments
PATCH /api/delivery/shipments/{shipmentId}/status	DELIVERY_AGENT	Update allowed package status
POST /api/delivery/shipments/{shipmentId}/verify-delivery	DELIVERY_AGENT	Verify OTP and complete delivery

```
POST /api/admin/shipments/701/assign
{ "deliveryAgentId": 8 }

PATCH /api/delivery/shipments/701/status
{
  "status": "OUT_FOR_DELIVERY",
  "notes": "Package is with delivery agent"
}

POST /api/delivery/shipments/701/verify-delivery
{ "recipientOtp": "******" }
```

Do not return OTP in API responses or logs. The customer-facing API may trigger OTP delivery to the verified phone/email; the delivery agent submits it only at completion.

9.5 Cancellation, returns and refunds

Method / endpoint	Role	Purpose
POST /api/orders/{orderId}/cancellations	CUSTOMER	Cancel all eligible items or selected item IDs
POST /api/returns	CUSTOMER	Create return request for delivered item quantity
PATCH /api/seller/returns/{returnCode}/decision	SELLER	Approve/reject own seller's return
POST /api/seller/returns/{returnCode}/inspection	SELLER	Record received condition and restockability
GET /api/orders/{orderId}/refunds	CUSTOMER (own order), ADMIN	View refund status

```
POST /api/orders/501/cancellations
{
  "orderItemIds": [402],
  "reason": "CUSTOMER_REQUEST"
}

POST /api/returns
{
  "orderItemId": 403,
  "quantity": 1,
  "reason": "DEFECTIVE",
  "description": "Product arrived damaged"
}

PATCH /api/seller/returns/RET-2026-8812/decision
{ "decision": "APPROVE", "reason": null }

POST /api/seller/returns/RET-2026-8812/inspection
{
  "condition": "RESTOCKABLE",
  "restockable": true,
  "notes": "Inspected; resale condition confirmed"
}
```

Inspection completion triggers the refund workflow through the shared refund service. The seller API must not send a gateway refund directly or mark a refund COMPLETED.

9.6 Core customer APIs (Member 1)

Method / endpoint	Role	Purpose
POST /api/auth/register; POST /api/auth/login	PUBLIC	Register/login; return authenticated token/session
GET /api/products; GET /api/products/{id}	PUBLIC	Browse catalogue and details
GET/POST/PUT/DELETE /api/cart/**	CUSTOMER	Own cart operations
POST /api/checkout	CUSTOMER	Validate cart, reserve stock and create checkout
POST /api/payments/{orderId}/initiate	CUSTOMER	Start payment for own order

Method / endpoint	Role	Purpose
POST /api/payments/webhook	Gateway signature verified	Verify payment event; commit reservation and advance order
GET /api/orders; GET /api/orders/{id}	CUSTOMER	View own orders and item/shipment progress

10. Notifications and event triggers

Event	Recipient(s)	Trigger
PAYMENT_CONFIRMED	Customer	Verified payment captured and reservation committed
SELLER_ACCEPTED / SELLER_REJECTED	Customer; seller audit	Seller decision recorded
PACKAGE_DISPATCHED	Customer	Shipment handover confirmed
OUT_FOR_DELIVERY	Customer	Shipment enters out-for-delivery
DELIVERED	Customer	OTP verified and shipment completed
CANCELLATION_UPDATED	Customer	Cancellation accepted/rejected and refund initiated if applicable
RETURN_UPDATED	Customer; seller	Return request decision, pickup, receipt and inspection
REFUND_UPDATED	Customer	Refund initiated, processing, completed or failed

Create notifications from committed domain events after the database transaction succeeds (transactional outbox is recommended). Notifications are informational; they must not be the source of truth for order, shipment or payment state.

11. Error handling, idempotency and transaction rules

11.1 Common response envelope

```
{
  "success": true,
  "message": "Human-readable result",
  "data": { },
  "timestamp": "2026-11-08T12:00:00Z"
}

Error example:
{
  "success": false,
  "message": "Insufficient stock",
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "details": [
      { "productId": 101, "requested": 5, "available": 2 }
    ]
  },
  "timestamp": "2026-11-08T12:00:00Z"
}
```

HTTP status	Use
400 Bad Request	Malformed payload, invalid field or invalid transition request
401 Unauthorized	Missing/invalid authentication
403 Forbidden	Authenticated role lacks permission
404 Not Found	Resource does not exist or is intentionally hidden due to ownership
409 Conflict	Insufficient stock, stale state, duplicate conflicting operation or usage limit reached
422 Unprocessable Entity	Business-rule validation failure such as return-window ineligibility
500 Internal Server Error	Unexpected server failure; do not expose stack traces

11.2 Idempotency and consistency

• Payment webhook: unique gateway event ID; verify signature; process once; duplicate event returns success with existing state.

• Inventory reserve/commit/release: unique checkout/reservation/event references; repeat calls do not change stock twice.

• Coupon redemption: lock coupon/usage rows or use atomic conditional update to enforce global and per-customer limits.

• Refund: unique idempotency_key and cumulative-refund check under transaction/lock.

• Status changes: validate current state and actor; write status history in the same transaction.

• External gateway calls: do not hold database row locks during long network calls. Persist intent, call gateway, then reconcile verified result.

• Use optimistic @Version or pessimistic row locking where appropriate; all multi-product reservation locks must use deterministic ordering.

12. Member responsibilities, integration sequence and sign-off

Sequence	Owais — Member 1	Rashika — Member 2	Exit condition
A. Contract sign-off	Revise shared schema/API docs; agree enums, DTOs and security rules	Review and confirm operational contracts	Both members approve same Version 2 contract
B. Shared foundation	Implement core models, auth, repositories, common response/error handling and integration points on feature/owais-core	Prepare operations module against agreed interfaces; no duplicate shared entities	Build/tests pass; foundation PR reviewed and merged
C. Operations module	Provide stable shared services and integration support	Implement seller inventory, coupons, fulfilment, shipments, delivery and return operations on feature/rashika-operations	Operations PR passes contract and ownership tests
D. End-to-end integration	Integrate cart, checkout, payment, customer order views and shared services	Integrate operations UI/APIs and notification events	Full multi-seller scenario works and tests pass

Joint sign-off checklist

• [ ] Entity fields and relationships accepted, especially Shipment/ShipmentItem and item-level discount snapshots.

• [ ] Order, OrderItem, Payment, Reservation, Shipment, Return and Refund enums accepted.

• [ ] Status transition matrix and order aggregation precedence accepted.

• [ ] Coupon ownership (ADMIN-managed for MVP unless changed), usage timing and allocation rounding accepted.

• [ ] Return eligibility, shipping/tax refund treatment and inspection deduction policy confirmed.

• [ ] API routes, request/response DTOs and common error envelope accepted.

• [ ] Role permissions and ownership enforcement accepted.

• [ ] Both members agree that this document is the integration source of truth.

Approval record

Member	Decision	Date / notes
Mohammed Owais (Member 1)	Pending confirmation	
Rashika (Member 2)	Pending final review	

Document control: Version 2.0 is a consolidated design proposal prepared from Rashika's review. It is not evidence that any API, entity, shipment workflow, coupon service or refund integration has been implemented. After both members approve, the agreed contract should be saved in the repository (for example, docs/architecture/shared-architecture-api-contract-v2.md) and used as the implementation reference.